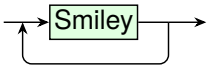
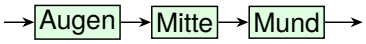
1.4 Die EBNF

Die **EBNF** (erweiterte Backus-Naur-Form) schreibt Produktionsregeln als Text, hier die Smileys aus 1.3:

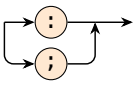
Smileyfolge



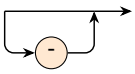
Smiley



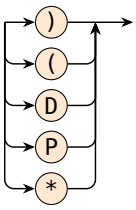
Augen



Mitte



Mund




```
R1 Smileyfolge = Smiley {Smiley}.
R2 Smiley = Augen Mitte Mund.
R3 Augen = ":" | ";".
R4 Mitte = ["-"].
R5 Mund = ")" | "(" | "D" | "P" | "*".
```

Die EBNF (vergleiche jede Regel mit ihrem Diagramm und ergänze)

`Name = … .`	Produktionsregel für ein Nichtterminal; `.` = Ende	`"D"`	Terminal (Zeichen des Alphabets)
`Mund`	Nichtterminal (Name einer Regel)	`A B`	Aneinanderreihung: erst A, dann B
`A | B`	Alternative: A oder B	`[A]`	Option: A einmal oder keinmal
`{A}`	Wiederholung: A 0-mal oder öfter	`(A | B) C`	Klammern: `("a"|"b")"c"` → `ac, bc`

Aufgabe 1 *Zeichen der EBNF*

a) Warum heißt R1 `Smiley {Smiley}` und nicht nur `{Smiley}`?

{…} erlaubt auch *keinmal*: Dann wäre ε eine Smileyfolge. Verlangt ist mindestens ein Smiley.

b) Schreibe R4 ohne eckige Klammern: `Mitte =` ε `| "-".`

Aufgabe 2 *Ableitung*

Ableitung von `;D:-(`: Je Schritt wird **ein** Nichtterminal nach seiner Produktionsregel ersetzt.

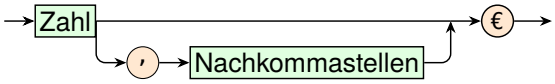
```
   Smileyfolge
R1 ⟹ Smiley Smiley
R2 ⟹ Augen Mitte Mund Smiley
R3 ⟹ ";" Mitte Mund Smiley
R4 ⟹ ";" Mund Smiley      (Mitte → ε)
R5 ⟹ ";" "D" Smiley
```

```
R2 ⟹ ";" "D" Augen Mitte Mund
R3 ⟹ ";" "D" ":" Mitte Mund
R4 ⟹ ";" "D" ":" "-" Mund
R5 ⟹ ";" "D" ":" "-" "("
```

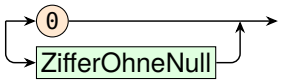
Aufgabe 3 *Vom Diagramm zur EBNF: Geldbeträge*

Gib die Grammatik der Beträge (`12€`, `0,79€`, nicht `02,99€`) in EBNF an. `ZifferOhneNull` ist gegeben.

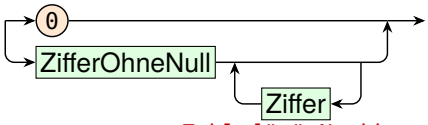
Geldbetrag



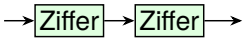
Ziffer



Zahl



Nachkommastellen




```
Geldbetrag = Zahl ["," Nachkommastellen] "€".
     Zahl = "0" | ZifferOhneNull {Ziffer}.
Nachkommastellen = Ziffer Ziffer.        Ziffer = "0" | ZifferOhneNull.
```

Aufgabe 4 *Regeln lesen und ändern: Molkereinummern*

Ein Code auf Milchprodukten gibt die Molkerei an (Leerzeichen nur zur Lesbarkeit):

R1 `Molkereinummer = "DE" Bundesland Betriebsnummer [Europa].`
R2 `Bundesland = "BB" | "BE" | "BW" | "BY" | … | "HH" | … | "RP" | … | "TH".` (16 Länder)
R3 `Betriebsnummer = Ziffer Ziffer Ziffer | Ziffer Ziffer Ziffer Ziffer Ziffer.`
R4 `Europa = "EG" | "EU".` R5 `Ziffer = "0" | "1" | … | "9".`

a) Gültig? Begründe bei ungültigen Codes.

`DE BY 13049 EG` gültig `DE HH 2514 EU` nein: 4 Ziffern

`BY 70123` nein: `DE` fehlt

b) Leite `DE RP 927` schrittweise ab.

`Molkereinummer` $\overset{R1}{\Longrightarrow}$ `"DE" Bundesland Betriebsnummer` $\overset{R2}{\Longrightarrow}$ `"DE" "RP" Betriebsnummer`
$\overset{R3}{\Longrightarrow}$ `"DE" "RP" Ziffer Ziffer Ziffer` $\overset{3\times R5}{\Longrightarrow}$ `"DE" "RP" "9" "2" "7"` (ohne `Europa`)

c) In Bayern gibt die erste Ziffer der Betriebsnummer den Regierungsbezirk an: nur `1` bis `7` (1 Oberbayern, 2 Niederbayern, …). Ändere und ergänze die Regeln.

R1 `Molkereinummer = "DE" (Bundesland Betriebsnummer | "BY" BetriebsnummerBY) [Europa].`
R2 wie bisher, aber *ohne* `"BY"`.
R6 `BetriebsnummerBY = Bezirk Ziffer Ziffer | Bezirk Ziffer Ziffer Ziffer Ziffer.`
R7 `Bezirk = "1" | "2" | "3" | "4" | "5" | "6" | "7".`

Aufgabe 5 *Selbst bauen: Uhrzeiten*

Uhrzeiten haben die Form `hh:mm` oder `hh:mm:ss`, immer zweistellig, von `00:00` bis `23:59:59`.

a) Drei ungültige Uhrzeiten, die knapp an einer Grenze scheitern: `24:00`, `19:60`, `12:30:75`

b) Gib eine Grammatik in EBNF an. *Tipp:* Welche Ziffern sind an welcher Stelle erlaubt? Die Stunden ab 20 sind ein Sonderfall.

```
Uhrzeit = Stunde ":" Minute [":" Minute].   (Sekunden sind wie Minuten aufgebaut)
Stunde = ("0" | "1") Ziffer | "2" ("0" | "1" | "2" | "3").
Minute = ("0" | "1" | "2" | "3" | "4" | "5") Ziffer.
Ziffer = "0" | "1" | "2" | "3" | "4" | "5" | "6" | "7" | "8" | "9".
```

Aufgabe 6 *Rekursion in der EBNF: die Terme aus 1.1*

In 1.1 (Aufgabe 5) war `sin cos` kein gültiger Term. Diese Grammatik legt die Terme fest:

R1 `Term = Summand {"+" Summand}.` R3 `Funktion = "sin" | "cos" | "sqrt".`
R2 `Summand = "x" | Funktion "(" Term ")".`

a) Gültig? Begründe bei ungültigen Wörtern.

`sqrt(x+x)` ja `cos(sin(x))+x` ja `sin(x)+` nein: Summand fehlt

`(x+x)` nein: `(` ohne Funktion

b) Welche Regeln verweisen über einen Umweg auf sich selbst? R1, R2: `Term` → `Summand` → `Term`

c) (mündlich) Warum kommt man hier ohne Rekursion nicht aus? Vergleiche mit 1.3, Aufgabe 5d.

d) Ändere R2, sodass auch `(x+x)` gültig ist:

`Summand = "x" | [Funktion] "(" Term ")".`